Алгоритмы

Алгоритмы

В разделе Архитектура мы разобрали, как организовать код: MVC, ECS, события, конечные автоматы. Это определяет *структуру* проекта. Но структура без содержания - пустая коробка. Нужны конкретные алгоритмы, которые решают конкретные задачи: найти путь, сгенерировать уровень, обнаружить столкновение.

Ну, вот об этом и поговорим.

Теперь перейдем к алгоритмам. Этот раздел подразумевается скорее как набор различных алгоритмов, использующихся во всевозможных игровых фичах. Т.е. если вы видите здесь, условно, алгоритм поиска путей A* (A-Star) - это не значит, что он самый эффективный/популярный/лучший, это пример алгоритма поиска путей - т.е. алгоритм, который решает проблему нахождения маршрута для человечка из точки A в точку B.

От идеи к алгоритмам

Зачем вам в игре алгоритмы?

- Появляется идея игры
- Появляется идея фичи

Возьмем, к примеру, врагов, которые атакуют игрока. Враги - шарики, которые катятся к игроку, при прикосновении наносят ему урон.

Допустим, мы знаем, как именно шарики наносят урон, как передвигаются, как "видят" игрока (наверное, мы не хотим, чтобы они видели его за километр и через стены).

Это вопросы нюансов реализации (например, тут у нас мог бы использоваться компонент ...). Но как шарикам пройти к игроку? Если у вас нет препятствий, то все легко - по прямой. А если есть препятствия? Например, двери. Некоторые двери закрыты, некоторые - нет. Как шарику понять, что надо идти влево 10 шагов, потом направо 5 и войти в открытую дверь? В этом нам помогает алгоритм поиска путей.

Так вот, алгоритмы в игре - это не самоцель. Как и все программирование. Они решают какую-то задачу, проблему. И если проблемы нет, то непонятно, какой алгоритм использовать.

А когда есть проблема - начинаешь искать информацию, погружаться в контекст, находить решения, анализировать их, сравнивать, выбирать лучшее.

От алгоритмов к идее

Однако, можно подойти с другой стороны. У нас есть алгоритм поиска путей, который позволяет найти маршрут из точки A в точку B. Где мы можем его использовать? Например, если мы добавим врагов, которые должны бегать за героем, нам потребуется этот алгоритм. Если у нас рандомно генерирующиеся локации, то нам потребуется процедурная генерация для уровней и т.д.

Т.е. оба направления работают: от проблемы к алгоритму и от алгоритма к идее. Можно выбрать интересный алгоритм и подумать, в какой игре он пригодится. Можно придумать фичу и искать алгоритм, который её реализует.

Какие алгоритмы пригодятся?

Ниже - обзор основных тем, с которыми вы, вероятно, столкнётесь при разработке игры. Каждая тема раскрыта в отдельном разделе.

Сложность в таблице примерная, приведена для ориентира. Любой алгоритм можно реализовать некорректно, топорно, превратив его в, условно, легкий (для реализации, понимания), или оптимизировать, скомбинировать с другими, улучшить, получив посложнее вариант.

И да, имейте в виду, что просто взять отсюда готовый или почти готовый вариант и получить за него баллы не получится.

Сложность алгоритмов

Алгоритм	Сложность	Где
AABB / circle overlap	Ультра-Легко	Коллизии
Sprite sheet indexing (`col = index % cols`)	Ультра-Легко	Анимации
Camera offset + clamp	Ультра-Легко	Камера
FSM (enum + switch)	Ультра-Легко	Конечные автоматы
Observer / Event pattern	Ультра-Легко	Событийная модель
Fisher-Yates shuffle	Ультра-Легко	Случайность
Shuffle bag	Ультра-Легко	Случайность

Алгоритм	Сложность	Где
Flood fill / BFS (валидация уровня)	Легко	Процедурная генерация
Random walk	Легко	Случайное блуждание
Cellular automata smoothing	Легко	Клеточные автоматы
Fixed timestep accumulator	Легко	Порядок обновления
Command + undo/redo (two stacks)	Легко	Пошаговые игры
Object pool	Легко	Системы частиц
Circular buffer (FPS counter)	Легко	Отладка
Seek / Flee / Arrive (steering)	Легко	Управляющие силы и Boids
Game loop + deltaTime	Легко	Основной игровой цикл
Patrol + LOS detection + hysteresis	Легко	Простой ИИ врагов
A* (heuristic search, priority queue)	Легко	Поиск путей
Priority queue / heap (initiative)	Легко	Пошаговые игры
Weighted random + binary search	Легко	Случайность
Box-Muller (normal distribution)	Легко	Случайность
Uniform Grid (bucket array)	Средне	Пространственные структуры
Swept collision (CCD)	Средне	Порядок обновления
Behavior tree tick traversal	Средне	Деревья поведений
Spatial hashing	Средне	Пространственные структуры
PRD (pseudo-random distribution)	Средне	Случайность
Minimax + alpha-beta pruning	Средне	Минимакс

Алгоритм	Сложность	Где
Boids (separation + alignment + cohesion)	Средне	Управляющие силы и Boids
Wave Function Collapse (constraint propagation)	Средне	WFC
L-система (expand + turtle + стек)	Средне	L-системы
Цепь Маркова (матрица переходов, взвешенный выбор)	Средне	Цепи Маркова
QuadTree (recursive subdivision)	Средне	Пространственные структуры
BSP dungeon generation (split + rooms + corridors)	Сложно	BSP-генерация
Lightmap + multiply blending (шейдеры, render targets, gamma)	Сложно	Простое освещение
Perlin noise (gradient dot products, fade curves)	Сложно	Шум Перлина
Client-side prediction + reconciliation	Сложно	Сетевая архитектура
Генетический алгоритм (популяция, отбор, скрещивание, мутация)	Сложно	Генетические алгоритмы
Нейросеть (перцептрон, обратное распространение)	Сложно	Нейросети для игр

Уровни, тайлы и поиск путей

Как представить уровень в программе? Как хранить ландшафт? Как найти маршрут для персонажа из точки A в точку B?

Мы разбиваем уровень на тайлы (плитки), получаем матрицу, из неё строим граф и применяем к нему алгоритмы поиска путей - от знакомого вам алгоритма Дейкстры до более эффективного при поиске до одной цели A*.

Подробнее: Уровни и поиск путей

Коллизии

Как понять, что два объекта столкнулись? Персонаж уперся в стену, снаряд попал во врага, игрок подобрал предмет - все это коллизии.

Проверять пересечение каждого пикселя с каждым - слишком дорого. Поэтому мы упрощаем объекты до примитивов (хитбоксов) - прямоугольников и окружностей - и считаем столкновения с ними. AABB, OBB, тайловые коллизии, пространственные структуры (BSP, QuadTree).

Подробнее: Коллизии

Пространственные структуры данных

Наивная проверка коллизий - каждый объект с каждым - это $O(n^2)$. При 500 объектах - 125 тысяч проверок каждый кадр. Как проверять только тех, кто рядом?

Разделяем мир на области: сетка (Uniform Grid), дерево квадрантов (QuadTree), пространственный хеш. Все три реализуют один интерфейс (insert + query), но отличаются поведением при разном распределении объектов. Фундаментальная идея, которая проявляется от игровых коллизий до KNN-поиска в машинном обучении.

Подробнее: Пространственные структуры данных

Процедурная генерация

Как создавать уровни алгоритмически, а не вручную? Актуально для рогаликов и любых игр, где уровни должны отличаться при каждом прохождении.

Несколько подходов, каждый со своим характером результата:

- **Случайное блуждание** - произвольные коридоры и тоннели, простейший алгоритм.
- **BSP-генерация** - структурированные подземелья с комнатами и коридорами.
- **Клеточные автоматы** - гладкие естественные пещеры из хаоса.
- **Шум Перлина** - плавный ландшафт, биомы, карты высот.
- **Wave Function Collapse** - тайлы с правилами соседства, алгоритм "схлопывает" ячейки с минимальной энтропией.
- **L-системы** - формальная грамматика + turtle-графика: из нескольких правил замены вырастают фрактальные деревья и кусты.

Процедурная генерация, кстати, может использоваться для всех аспектов игры: музыка, сюжет, персонажи, предметы и т.д.

Подробнее: Процедурная генерация

Камера и отсечение

Уровень больше экрана. Как показать игроку нужную часть мира? Как не рисовать то, что за пределами экрана?

Камера как смещение координат, плавное следование за игроком, ограничение по краям уровня, отсечение невидимых тайлов и объектов.

Подробнее: Камера и отсечение

Конечные автоматы

Как управлять состояниями персонажа, не утонув в десятках флагов и if-else? Как организовать переходы между сценами игры?

Конечный автомат - всегда в одном состоянии, переходы четко определены. Работает и для персонажей, и для менеджера сцен.

Подробнее: Конечные автоматы

Событийная модель

Как передавать информацию между системами, не связывая их напрямую? Как сделать так, чтобы InputManager не знал обо всех объектах в игре?

Паттерн Observer / pub-sub: издатель объявляет о событии, подписчики реагируют. Простой класс Event, событийная шина, встроенные события C#.

Подробнее: Событийная модель

Анимации

Как анимировать персонажа? Как переключаться между бегом, атакой и стоянием? Как привязать событие к определённому кадру?

Спрайт-листы, покадровая анимация через deltaTime, контроллер анимаций, одноразовые анимации (атака, смерть), отзеркаливание спрайта, события на кадрах.

Подробнее: Анимации

Деревья поведений (Behavior Trees)

Конечный автомат работает для 3-5 состояний. А если враг должен патрулировать, слышать шум, преследовать, атаковать, звать подкрепление и убегать? FSM превращается в паутину переходов.

Behavior tree - дерево приоритетов: Sequence (И), Selector (ИЛИ), листья-действия. Добавить поведение = добавить ветку, не трогая остальные.

Подробнее: Деревья поведений

Простой ИИ врагов

Как сделать, чтобы враг патрулировал, замечал игрока, бежал к нему и атаковал? Как не дёргаться между "вижу/не вижу"?

Патрулирование по точкам, обнаружение (радиус + луч видимости), преследование (напрямую или через A*), атака с кулдауном, FSM врага (Patrol -> Chase -> Attack), гистерезис.

Подробнее: Простой ИИ врагов

Управляющие силы и стайное поведение (Boids)

Враг бежит к игроку по прямой - как робот. Как добавить плавность, уклонение, блуждание? Как сделать стаю летучих мышей, косяк рыб или толпу зомби?

Управляющие силы (steering behaviors) - каждое поведение возвращает вектор, мы складываем их и получаем направление движения. Seek, Flee, Arrive, Wander, Pursuit. Boids Крейга Рейнольдса: три правила (разделение, выравнивание, сплочённость) -> из простых правил возникает стая.

Подробнее: Управляющие силы и Boids

Системы частиц

Как сделать огонь, дым, искры, кровь, дождь? Как обработать тысячи мелких объектов без тормозов?

Частица как набор данных, эмиттер, burst-спавн, гравитация/затухание/прозрачность, пул объектов, data-oriented design для производительности.

Подробнее: Системы частиц

Простое освещение

Как сделать тёмные подземелья, фонарик в руке, ночные уровни с факелами? Как реализовать смену дня и ночи?

Карта освещения (light map) с multiply-блендингом, градиентные текстуры, несколько источников, цикл день/ночь, идея рейкастинга для теней.

Подробнее: Простое освещение

Минимакс и деревья игр

Как сделать ИИ для пошаговой игры? Как научить босса выбирать оптимальное действие?

Дерево возможных ходов, алгоритм минимакс, альфа-бета отсечение, ограничение глубины с эвристикой. Примеры: крестики-нолики, боссфайт с Attack/Heal/Heavy Attack.

Подробнее: Минимакс и деревья игр

Генетические алгоритмы

Как подобрать параметры для 50 типов врагов, не настраивая каждого вручную? Как найти хорошее решение в огромном пространстве вариантов, где нет формулы?

Генетический алгоритм поддерживает популяцию кандидатов, оценивает каждого, отбирает лучших, скрещивает и мутирует - и через десятки поколений находит хорошее решение. Тот же подход работает для балансировки оружия, генерации уровней, подбора цветовых палитр.

Подробнее: Генетические алгоритмы

Нейросети для игр

Руками написанные правила (автоматы, деревья поведений) работают, пока правил немного. Нейросеть может *выучить* правила из данных или опыта: подать на вход состояние игры, получить на выходе действие - и подстроить веса так, чтобы действия были разумными.
В разумных, конечно, пределах.

Подробнее: Нейросети для игр

Основной игровой цикл

Как вообще работает игра? Почему она не "зависает" между кадрами? Что такое deltaTime и зачем он нужен?

Игровой цикл, кадры, реальное и игровое время, пауза, замедление - основа, на которой держится всё остальное.

Подробнее: Основной игровой цикл

Порядок обновления и физический цикл

В каком порядке обновлять системы внутри `update()` ? Почему физика "ломается" при переменном FPS? Как объект проходит сквозь стену?

Порядок систем (Input -> AI -> Movement -> Collision -> Logic -> Cleanup), фиксированный шаг с аккумулятором, туннелирование (swept collision), zombie-обновления, фазы коллизий.

Подробнее: Порядок обновления и физический цикл

Случайность и вероятность

Как сделать "честный" рандом? Почему при шансе 25% игрок может промахнуться 10 раз подряд - и как это исправить?

Псевдослучайность и seed, лут-таблицы (взвешенный выбор с бинарным поиском), pseudo-random distribution (Dota 2), shuffle bag (Тетрис), нормальное распределение для урона, управление seed'ами.

Подробнее: Случайность и вероятность

Цепи Маркова

Конечный автомат с вероятностными переходами. Вместо "если видит игрока - переход в Chase" - "с вероятностью 30% Chase, 50% продолжать Patrol, 20% Flee". Матрица переходов, взвешенный выбор, стационарное распределение.

Тот же формализм используется для генерации имён (биграммы), текстов, музыки. А в теории - для PageRank, распознавания речи и reinforcement learning (MDP).

Подробнее: Цепи Маркова

Пошаговые игры и паттерн Command

Как устроен цикл пошаговой игры? Как реализовать отмену хода? Как ИИ "думает" над ходом?

Пошаговый цикл как FSM, паттерн Command (execute/undo), undo/redo на двух стеках, очередь действий, инициатива через приоритетную очередь, реплей и lockstep.

Подробнее: Пошаговые игры и паттерн Command

Отладка и профилирование

Игра тормозит - как понять, почему? Враг ведёт себя странно - как увидеть его логику?

Встроенные инструменты разработчика: визуализация хитбоксов и ИИ-состояний, FPS-график на кольцевом буфере, профайлер по системам, замедление и покадровый шаг, Big-O на практике.

Подробнее: Отладка и инструменты разработчика

Сетевая архитектура

Как два компьютера видят один игровой мир? Как передавать состояние по сети без тормозов? Как скрыть задержку в 100 мс?

Сериализация данных (struct.pack, бинарная упаковка, дельта-компрессия), авторитарный сервер (клиент отправляет ввод, сервер решает), клиентское предсказание с серверной коррекцией, интерполяция чужих игроков.

Подробнее: Сетевая архитектура

Что выбрать?

Есть враги, которые бегают за игроком? Нужен поиск путей. Рандомные уровни? Процедурная генерация. Большой уровень? Камера и отсечение. Много состояний у персонажа? Конечные автоматы.

А если вы ещё не определились с фичами - пролистайте разделы выше, возможно, какой-то алгоритм подскажет идею. Или загляните в раздел Выбор жанра и идеи - там разобраны варианты и что за каждым из них стоит.

Помимо алгоритмов, стоит продумать и структуру кода. Обзор архитектурных подходов - MVC, ECS, событийная модель, наследование и его проблемы - в разделе Архитектура.

Powered by Wiki.js